

```
import random
import time
import math
```

✓ Advanced Divide and Conquer Algorithms

✓ 1. Strassen Matrix Multiplication

```
def standard_multiply(A, B):
    rows_A = len(A)
    cols_A = len(A[0])
    rows_B = len(B)
    cols_B = len(B[0])

    if cols_A != rows_B:
        raise ValueError("Incompatible matrix dimensions")

    C = [[0] * cols_B for _ in range(rows_A)]

    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(cols_A):
                C[i][j] += A[i][k] * B[k][j]

    return C

def add_matrix(A, B):
    n = len(A)
    return [[A[i][j] + B[i][j] for j in range(n)] for i in range(n)]

def subtract_matrix(A, B):
    n = len(A)
    return [[A[i][j] - B[i][j] for j in range(n)] for i in range(n)]

def strassen_recursive(A, B, threshold=1):
    n = len(A)

    if n <= threshold:
        return standard_multiply(A, B)

    if n == 1:
        return [[A[0][0] * B[0][0]]]

    mid = n // 2
```

```
A11 = [row[:mid] for row in A[:mid]]
A12 = [row[mid:] for row in A[:mid]]
A21 = [row[:mid] for row in A[mid:]]
A22 = [row[mid:] for row in A[mid:]]

B11 = [row[:mid] for row in B[:mid]]
B12 = [row[mid:] for row in B[:mid]]
B21 = [row[:mid] for row in B[mid:]]
B22 = [row[mid:] for row in B[mid:]]

M1 = strassen_recursive(
    add_matrix(A11, A22),
    add_matrix(B11, B22),
    threshold
)

M2 = strassen_recursive(
    add_matrix(A21, A22),
    B11,
    threshold
)

M3 = strassen_recursive(
    A11,
    subtract_matrix(B12, B22),
    threshold
)

M4 = strassen_recursive(
    A22,
    subtract_matrix(B21, B11),
    threshold
)

M5 = strassen_recursive(
    add_matrix(A11, A12),
    B22,
    threshold
)

M6 = strassen_recursive(
    subtract_matrix(A21, A11),
    add_matrix(B11, B12),
    threshold
)

M7 = strassen_recursive(
    subtract_matrix(A12, A22),
    add_matrix(B21, B22),
    threshold
)

C11 = add_matrix(
    subtract_matrix(add_matrix(M1, M4), M5),
    M7
```

```

    )

    C12 = add_matrix(M3, M5)

    C21 = add_matrix(M2, M4)

    C22 = add_matrix(
        subtract_matrix(add_matrix(M1, M3), M2),
        M6
    )

    C = []

    for i in range(mid):
        C.append(C11[i] + C12[i])

    for i in range(mid):
        C.append(C21[i] + C22[i])

    return C

def next_power_of_two(n):
    size = 1
    while size < n:
        size *= 2
    return size

def strassen_multiply(A, B, threshold=1):
    if len(A[0]) != len(B):
        raise ValueError("Incompatible matrix dimensions")

    rows_A = len(A)
    cols_A = len(A[0])
    rows_B = len(B)
    cols_B = len(B[0])

    size = next_power_of_two(
        max(rows_A, cols_A, rows_B, cols_B)
    )

    padded_A = [
        row + [0] * (size - cols_A)
        for row in A
    ]

    while len(padded_A) < size:
        padded_A.append([0] * size)

    padded_B = [
        row + [0] * (size - cols_B)
        for row in B
    ]

```

```
while len(padded_B) < size:
    padded_B.append([0] * size)

result = strassen_recursive(
    padded_A,
    padded_B,
    threshold
)

return [
    row[:cols_B]
    for row in result[:rows_A]
]

def count_multiplications_standard(n):
    return n ** 3

def count_multiplications_strassen(n):
    if n <= 1:
        return 1

    size = next_power_of_two(n)
    levels = size.bit_length() - 1

    return 7 ** levels

print("=" * 60)
print("1. STRASSEN MATRIX MULTIPLICATION TEST CASES")
print("=" * 60)

A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]

print("A =", A)
print("B =", B)

print("Standard:")
print(standard_multiply(A, B))

print("Strassen:")
print(strassen_multiply(A, B))

assert strassen_multiply(A, B) == standard_multiply(A, B)

A3 = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

B3 = [
    [9, 8, 7],
```

```
[6, 5, 4],
[3, 2, 1]
]

assert strassen_multiply(A3, B3) == standard_multiply(A3, B3)

print("3x3 Test Passed")

A_rect = [
    [1, 2, 3],
    [4, 5, 6]
]

B_rect = [
    [7, 8],
    [9, 10],
    [11, 12]
]

assert strassen_multiply(A_rect, B_rect) == standard_multiply(A_rect, B_rect)

print("Rectangular Matrix Test Passed")

print("\nMultiplication Counts")

for n in [2, 4, 8, 16, 32, 64]:
    print(
        "n =", n,
        "| Standard =", count_multiplications_standard(n),
        "| Strassen =", count_multiplications_strassen(n)
    )

print("\nSTRASSEN BENCHMARK")

for n in [4, 8, 16]:
    A = [[random.randint(1, 10) for _ in range(n)] for _ in range(n)]
    B = [[random.randint(1, 10) for _ in range(n)] for _ in range(n)]

    start = time.perf_counter()
    standard_multiply(A, B)
    standard_time = time.perf_counter() - start

    start = time.perf_counter()
    strassen_multiply(A, B, threshold=2)
    strassen_time = time.perf_counter() - start

    print(
        f"Size {n}x{n}: "
        f"Standard = {standard_time:.6f}s, "
        f"Strassen = {strassen_time:.6f}s"
    )
```

```
print("\nHYBRID STRASSEN Test")

A = [[random.randint(-5, 5) for _ in range(4)] for _ in range(4)]
B = [[random.randint(-5, 5) for _ in range(4)] for _ in range(4)]

assert strassen_multiply(A, B, threshold=2) == standard_multiply(A, B)

print("Hybrid Strassen Test Passed")
```

```
=====
1. STRASSEN MATRIX MULTIPLICATION TEST CASES
=====
A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]
Standard:
[[19, 22], [43, 50]]
Strassen:
[[19, 22], [43, 50]]
3x3 Test Passed
Rectangular Matrix Test Passed

Multiplication Counts
n = 2 | Standard = 8 | Strassen = 7
n = 4 | Standard = 64 | Strassen = 49
n = 8 | Standard = 512 | Strassen = 343
n = 16 | Standard = 4096 | Strassen = 2401
n = 32 | Standard = 32768 | Strassen = 16807
n = 64 | Standard = 262144 | Strassen = 117649

STRASSEN BENCHMARK
Size 4x4: Standard = 0.000012s, Strassen = 0.000060s
Size 8x8: Standard = 0.000049s, Strassen = 0.000327s
Size 16x16: Standard = 0.000363s, Strassen = 0.002927s

HYBRID STRASSEN Test
Hybrid Strassen Test Passed
```

✓ 2. Karatsuba Multiplication

```
def karatsuba(x, y):
    if x == 0 or y == 0:
        return 0

    sign = -1 if (x < 0) ^ (y < 0) else 1

    x = abs(x)
    y = abs(y)

    if x < 10 or y < 10:
        return sign * (x * y)

    n = max(len(str(x)), len(str(y)))
    m = n // 2

    base = 10 ** m
```

```
high_x = x // base
low_x = x % base

high_y = y // base
low_y = y % base

z0 = karatsuba(low_x, low_y)
z2 = karatsuba(high_x, high_y)

z1 = karatsuba(
    low_x + high_x,
    low_y + high_y
) - z2 - z0

result = (
    z2 * base * base
    + z1 * base
    + z0
)

return sign * result

def karatsuba_with_count(x, y, counter):
    counter[0] += 1

    if x == 0 or y == 0:
        return 0

    sign = -1 if (x < 0) ^ (y < 0) else 1

    x = abs(x)
    y = abs(y)

    if x < 10 or y < 10:
        return sign * (x * y)

    n = max(len(str(x)), len(str(y)))
    m = n // 2

    base = 10 ** m

    high_x = x // base
    low_x = x % base

    high_y = y // base
    low_y = y % base

    z0 = karatsuba_with_count(
        low_x, low_y, counter
    )

    z2 = karatsuba_with_count(
        high_x, high_y, counter
```

```

    )

    z1 = karatsuba_with_count(
        low_x + high_x,
        low_y + high_y,
        counter
    ) - z2 - z0

    result = (
        z2 * base * base
        + z1 * base
        + z0
    )

    return sign * result

def karatsuba_traced(x, y, trace, depth=0):
    trace.append(
        (
            depth,
            x,
            y
        )
    )

    if x == 0 or y == 0:
        return 0

    sign = -1 if (x < 0) ^ (y < 0) else 1

    x = abs(x)
    y = abs(y)

    if x < 10 or y < 10:
        return sign * x * y

    n = max(len(str(x)), len(str(y)))
    m = n // 2
    base = 10 ** m

    high_x = x // base
    low_x = x % base

    high_y = y // base
    low_y = y % base

    z0 = karatsuba_traced(
        low_x,
        low_y,
        trace,
        depth + 1
    )

    z2 = karatsuba_traced(

```



```

        high_x,
        high_y,
        trace,
        depth + 1
    )

    z1 = karatsuba_traced(
        low_x + high_x,
        low_y + high_y,
        trace,
        depth + 1
    ) - z2 - z0

    return sign * (
        z2 * base * base
        + z1 * base
        + z0
    )

```

```

print("\n" + "=" * 60)
print("2. KARATSUBA MULTIPLICATION TEST CASES")
print("=" * 60)

test_cases = [
    (1234, 5678),
    (123456789, 987654321),
    (9, 9),
    (0, 12345)
]

for x, y in test_cases:
    result = karatsuba(x, y)

    print(
        f"{x} x {y} = {result}"
    )

    assert result == x * y

big1 = int("9" * 50)
big2 = int("8" * 50)

assert karatsuba(big1, big2) == big1 * big2

print("50-digit Test Passed")

print("\nRecursive Call Count")

counter = [0]

result = karatsuba_with_count(
    1234,
    5678,

```

```

        counter
    )

    print("Result =", result)
    print("Recursive Calls =", counter[0])

    assert result == 1234 * 5678

    counter2 = [0]

    karatsuba_with_count(
        9,
        9,
        counter2
    )

    assert counter2[0] == 1

    print("Base Case Call Test Passed")

    print("\nKaratsuba Trace")

    trace = []

    result, trace = (
        karatsuba_traced(1234, 56, trace),
        trace
    )

    print("Result =", result)
    print("Number of trace entries =", len(trace))

    for entry in trace[:10]:
        print(
            "Depth:",
            entry[0],
            "X:",
            entry[1],
            "Y:",
            entry[2]
        )

```

```

=====
2. KARATSUBA MULTIPLICATION TEST CASES
=====
1234 x 5678 = 7006652
123456789 x 987654321 = 121932631112635269
9 x 9 = 81
0 x 12345 = 0
50-digit Test Passed

Recursive Call Count
Result = 7006652

```

```

Recursive Calls = 16
Base Case Call Test Passed

Karatsuba Trace
Result = 69104
Number of trace entries = 13
Depth: 0 X: 1234 Y: 56
Depth: 1 X: 34 Y: 56
Depth: 2 X: 4 Y: 6
Depth: 2 X: 3 Y: 5
Depth: 2 X: 7 Y: 11
Depth: 1 X: 12 Y: 0
Depth: 1 X: 46 Y: 56
Depth: 2 X: 6 Y: 6
Depth: 2 X: 4 Y: 5
Depth: 2 X: 10 Y: 11

```

✓ 3. Karatsuba-style Polynomial Multiplication

```

def multiply_polynomials_naive(p, q):
    result = [0] * (len(p) + len(q) - 1)

    for i in range(len(p)):
        for j in range(len(q)):
            result[i + j] += p[i] * q[j]

    return result

def add_poly(a, b):
    n = max(len(a), len(b))
    result = [0] * n

    for i in range(len(a)):
        result[i] += a[i]

    for i in range(len(b)):
        result[i] += b[i]

    while len(result) > 1 and result[-1] == 0:
        result.pop()

    return result

def subtract_poly(a, b):
    n = max(len(a), len(b))
    result = [0] * n

    for i in range(len(a)):
        result[i] += a[i]

    for i in range(len(b)):
        result[i] -= b[i]

```

```
while len(result) > 1 and result[-1] == 0:
    result.pop()

return result

def karatsuba_poly(p, q):
    while len(p) > 1 and p[-1] == 0:
        p.pop()

    while len(q) > 1 and q[-1] == 0:
        q.pop()

    if not p or not q:
        return [0]

    if len(p) <= 2 or len(q) <= 2:
        return multiply_polynomials_naive(p, q)

    n = max(len(p), len(q))
    m = n // 2

    p_low = p[:m]
    p_high = p[m:]

    q_low = q[:m]
    q_high = q[m:]

    z0 = karatsuba_poly(
        p_low,
        q_low
    )

    z2 = karatsuba_poly(
        p_high,
        q_high
    )

    p_sum = add_poly(p_low, p_high)
    q_sum = add_poly(q_low, q_high)

    z1 = karatsuba_poly(
        p_sum,
        q_sum
    )

    z1 = subtract_poly(
        subtract_poly(z1, z2),
        z0
    )

    result = [0] * (
        max(
            len(z0),
```

```
        len(z1) + m,
        len(z2) + 2 * m
    )
)

for i, value in enumerate(z0):
    result[i] += value

for i, value in enumerate(z1):
    result[i + m] += value

for i, value in enumerate(z2):
    result[i + 2 * m] += value

while len(result) > 1 and result[-1] == 0:
    result.pop()

return result
```

```
print("\nPolynomial Multiplication")

p1 = [1, 2]
p2 = [3, 4]

print(
    "Naive:",
    multiply_polynomials_naive(p1, p2)
)

print(
    "Karatsuba:",
    karatsuba_poly(p1, p2)
)

assert karatsuba_poly(p1, p2) == [3, 10, 8]

p1 = [1, 2, 3, 4]
p2 = [5, 6, 7, 8]

naive_result = multiply_polynomials_naive(
    p1,
    p2
)

karatsuba_result = karatsuba_poly(
    p1,
    p2
)

assert karatsuba_result == naive_result

print("Polynomial Test Passed")
```

```
Polynomial Multiplication  
Naive: [3, 10, 8]  
Karatsuba: [3, 10, 8]  
Polynomial Test Passed
```

✓ 4. Closest Pair of Points - Divide and Conquer

```
def distance(p1, p2):  
    return math.sqrt(  
        (p1[0] - p2[0]) ** 2  
        + (p1[1] - p2[1]) ** 2  
    )  
  
def brute_force_closest(points):  
    min_dist = float("inf")  
    best_pair = None  
  
    n = len(points)  
  
    for i in range(n):  
        for j in range(i + 1, n):  
            d = distance(points[i], points[j])  
  
            if d < min_dist:  
                min_dist = d  
                best_pair = (  
                    points[i],  
                    points[j]  
                )  
  
    return best_pair, min_dist  
  
def closest_pair_recursive(points_x, points_y):  
    n = len(points_x)  
  
    if n <= 3:  
        return brute_force_closest(points_x)  
  
    mid = n // 2  
  
    left_x = points_x[:mid]  
    right_x = points_x[mid:]  
  
    mid_x = points_x[mid][0]  
  
    left_set = set(left_x)  
  
    left_y = []  
    right_y = []
```

```

for p in points_y:
    if p in left_set:
        left_y.append(p)
    else:
        right_y.append(p)

pair_left, dist_left = closest_pair_recursive(
    left_x,
    left_y
)

pair_right, dist_right = closest_pair_recursive(
    right_x,
    right_y
)

if dist_left < dist_right:
    best_pair = pair_left
    min_dist = dist_left
else:
    best_pair = pair_right
    min_dist = dist_right

strip = [
    p for p in points_y
    if abs(p[0] - mid_x) < min_dist
]

for i in range(len(strip)):
    j = i + 1

    while (
        j < len(strip)
        and (strip[j][1] - strip[i][1]) < min_dist
    ):
        d = distance(
            strip[i],
            strip[j]
        )

        if d < min_dist:
            min_dist = d
            best_pair = (
                strip[i],
                strip[j]
            )

        j += 1

    return best_pair, min_dist

def closest_pair_of_points(points):
    if len(points) < 2:
        return None, float("inf")

```

```
points_x = sorted(
    points,
    key=lambda p: p[0]
)

points_y = sorted(
    points,
    key=lambda p: p[1]
)

return closest_pair_recursive(
    points_x,
    points_y
)
```

```
print("\n" + "=" * 60)
print("3. CLOSEST PAIR OF POINTS TEST CASES")
print("=" * 60)

pts = [
    (2, 3),
    (12, 30),
    (40, 50),
    (5, 1),
    (12, 10),
    (3, 4)
]

pair, d = closest_pair_of_points(pts)

print("Closest Pair:", pair)
print("Minimum Distance:", d)

brute_pair, brute_d = brute_force_closest(pts)

assert abs(d - brute_d) < 1e-9

print("Correctness Test Passed")

random.seed(0)

random_pts = [
    (
        random.uniform(0, 100),
        random.uniform(0, 100)
    )
    for _ in range(50)
]

pair, d = closest_pair_of_points(
    random_pts
)
```



```
brute_pair, brute_d = brute_force_closest(
    random_pts
)

assert abs(d - brute_d) < 1e-9

print("50 Random Points Test Passed")

print("\nCOLLISION DETECTION")

def detect_potential_collision(
    sprites,
    threshold
):
    pair, min_dist = closest_pair_of_points(
        sprites
    )

    if min_dist <= threshold:
        return pair, min_dist

    return None, min_dist

sprites = [
    (0, 0),
    (1, 1),
    (50, 50),
    (100, 100),
    (1.2, 0.9)
]

pair, min_dist = detect_potential_collision(
    sprites,
    threshold=2.0
)

print("Pair:", pair)
print("Distance:", min_dist)

assert pair is not None
assert min_dist <= 2.0

far_sprites = [
    (0, 0),
    (100, 100),
    (200, 200)
]

pair2, min_dist2 = detect_potential_collision(
    far_sprites,
    threshold=1.0
```

```
)

assert pair2 is None

print("Collision Test Passed")

print("\nSENSOR NODE PLACEMENT")

def check_minimum_spacing(
    nodes,
    min_safe_distance
):
    pair, min_dist = closest_pair_of_points(
        nodes
    )

    ok = min_dist >= min_safe_distance

    return ok, pair, min_dist

nodes = [
    (0, 0),
    (10, 10),
    (10.5, 10.2),
    (30, 40),
    (31, 41)
]

ok, pair, min_dist = check_minimum_spacing(
    nodes,
    1.0
)

print("Safe:", ok)
print("Closest Pair:", pair)
print("Minimum Distance:", min_dist)

assert ok is False

spaced_nodes = [
    (0, 0),
    (10, 10),
    (20, 20),
    (30, 30)
]

ok2, _, _ = check_minimum_spacing(
    spaced_nodes,
    1.0
)

assert ok2 is True
```

```
print("Sensor Spacing Test Passed")
```

```
=====
3. CLOSEST PAIR OF POINTS TEST CASES
=====
```

```
Closest Pair: ((2, 3), (3, 4))
Minimum Distance: 1.4142135623730951
Correctness Test Passed
50 Random Points Test Passed
```

```
COLLISION DETECTION
```

```
Pair: ((1.2, 0.9), (1, 1))
Distance: 0.2236067977499789
Collision Test Passed
```

```
SENSOR NODE PLACEMENT
```

```
Safe: False
Closest Pair: ((10, 10), (10.5, 10.2))
Minimum Distance: 0.5385164807134502
Sensor Spacing Test Passed
```

✓ 5. Median of Medians

```
def median_of_medians_select(arr, k):
    if not arr:
        raise ValueError("Array cannot be empty")

    if k < 0 or k >= len(arr):
        raise IndexError("Invalid k")

    if len(arr) <= 5:
        return sorted(arr)[k]

    groups = []

    for i in range(0, len(arr), 5):
        group = arr[i:i + 5]
        group.sort()

        groups.append(
            group[len(group) // 2]
        )

    pivot = median_of_medians_select(
        groups,
        len(groups) // 2
    )

    low = [
        x for x in arr
        if x < pivot
    ]
```

```
equal = [  
    x for x in arr  
    if x == pivot  
]  
  
high = [  
    x for x in arr  
    if x > pivot  
]  
  
if k < len(low):  
    return median_of_medians_select(  
        low,  
        k  
    )  
  
elif k < len(low) + len(equal):  
    return pivot  
  
else:  
    return median_of_medians_select(  
        high,  
        k - len(low) - len(equal)  
    )  
  
def kth_smallest_delivery_time(  
    delivery_times,  
    k  
):  
    return median_of_medians_select(  
        delivery_times,  
        k  
    )
```

◆ Gemini

```
print("\n" + "=" * 60)  
print("4. MEDIAN OF MEDIANS TEST CASES")  
print("=" * 60)  
  
data = [  
    12,  
    3,  
    5,  
    7,  
    4,  
    19,  
    26  
]  
  
print("Data:", data)  
  
for k in range(len(data)):
```

```
    result = median_of_medians_select(
        data,
        k
    )

    print(
        f"k={k} -> {result}"
    )

    assert result == sorted(data)[k]

print("Basic Test Passed")

random.seed(1)

big_data = [
    random.randint(0, 10000)
    for _ in range(500)
]

sorted_big = sorted(big_data)

for k in [0, 10, 250, 499]:
    result = median_of_medians_select(
        big_data,
        k
    )

    print(
        f"Random Data k={k}: {result}"
    )

    assert result == sorted_big[k]

print("500 Element Random Test Passed")

print("\nK-th Fastest Delivery Time")

already_sorted = list(range(1, 501))

assert kth_smallest_delivery_time(
    already_sorted,
    0
) == 1

assert kth_smallest_delivery_time(
    already_sorted,
    499
) == 500

assert kth_smallest_delivery_time(
    already_sorted,
    250
```

```

) == 251

print("Sorted 500-element Test Passed")

delivery_times = [
    45,
    30,
    60,
    25,
    50,
    40,
    35,
    55,
    20,
    65
]

for k in range(len(delivery_times)):
    result = kth_smallest_delivery_time(
        delivery_times,
        k
    )

    print(
        f"{k + 1}th fastest = {result}"
    )

    assert result == sorted(delivery_times)[k]

```

```

=====
4. MEDIAN OF MEDIANS TEST CASES
=====
Data: [12, 3, 5, 7, 4, 19, 26]
k=0 -> 3
k=1 -> 4
k=2 -> 5
k=3 -> 7
k=4 -> 12
k=5 -> 19
k=6 -> 26
Basic Test Passed
Random Data k=0: 25
Random Data k=10: 202
Random Data k=250: 4970
Random Data k=499: 9976
500 Element Random Test Passed

K-th Fastest Delivery Time
Sorted 500-element Test Passed
1th fastest = 20
2th fastest = 25
3th fastest = 30
4th fastest = 35
5th fastest = 40
6th fastest = 45

```

```

7th fastest = 50
8th fastest = 55
9th fastest = 60
10th fastest = 65

```

✓ Complexity Summary

```

print("=" * 60)
print("COMPLEXITY SUMMARY")
print("=" * 60)

print("""
1. Strassen Matrix Multiplication
   Time  :  $O(n^{2.807})$ 
   Space :  $O(n^2)$ 

2. Standard Matrix Multiplication
   Time  :  $O(n^3)$ 
   Space :  $O(n^2)$ 

3. Karatsuba Multiplication
   Time  :  $O(n^{1.585})$ 
   Space :  $O(n)$  auxiliary recursion-related space

4. Schoolbook Multiplication
   Time  :  $O(n^2)$ 

5. Closest Pair - Divide and Conquer
   Time  :  $O(n \log n)$ 
   Space :  $O(n)$ 

6. Closest Pair - Brute Force
   Time  :  $O(n^2)$ 

7. Median of Medians
   Time  :  $O(n)$  worst case
   Space :  $O(n)$ 

8. Naive Quickselect
   Average :  $O(n)$ 
   Worst   :  $O(n^2)$ 
""")

print("=" * 60)
print("ALL TEST CASES COMPLETED SUCCESSFULLY!")
print("=" * 60)

```

```

=====
COMPLEXITY SUMMARY
=====

1. Strassen Matrix Multiplication
   Time  :  $O(n^{2.807})$ 
   Space :  $O(n^2)$ 

```

2. Standard Matrix Multiplication
Time : $O(n^3)$
Space : $O(n^2)$
3. Karatsuba Multiplication
Time : $O(n^{1.585})$
Space : $O(n)$ auxiliary recursion-related space
4. Schoolbook Multiplication
Time : $O(n^2)$
5. Closest Pair - Divide and Conquer
Time : $O(n \log n)$
Space : $O(n)$
6. Closest Pair - Brute Force
Time : $O(n^2)$
7. Median of Medians
Time : $O(n)$ worst case
Space : $O(n)$
8. Naive Quickselect
Average : $O(n)$
Worst : $O(n^2)$

=====
ALL TEST CASES COMPLETED SUCCESSFULLY!
=====